

FORMATION PYTHON

J5

Intégration et projets

argparse, logging, vectorisation, checklist qualité :
la chaîne complète en une commande — puis les soutenances.

10-integration

projet-final

Objectifs de la journée



La chaîne complète

lecture → validation → traitement → restitution : le modèle fourni, lu ensemble.



Un vrai outil en ligne de commande

argparse (types, défauts, exclusions), sys.exit, logging plutôt que print.



Du code qui tient la charge

Vectoriser plutôt que boucler ; la checklist qualité avant de rendre.



Finir et soutenir

Matinée : dernier jalon. Après-midi : 10 min de démo + 5 de questions.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

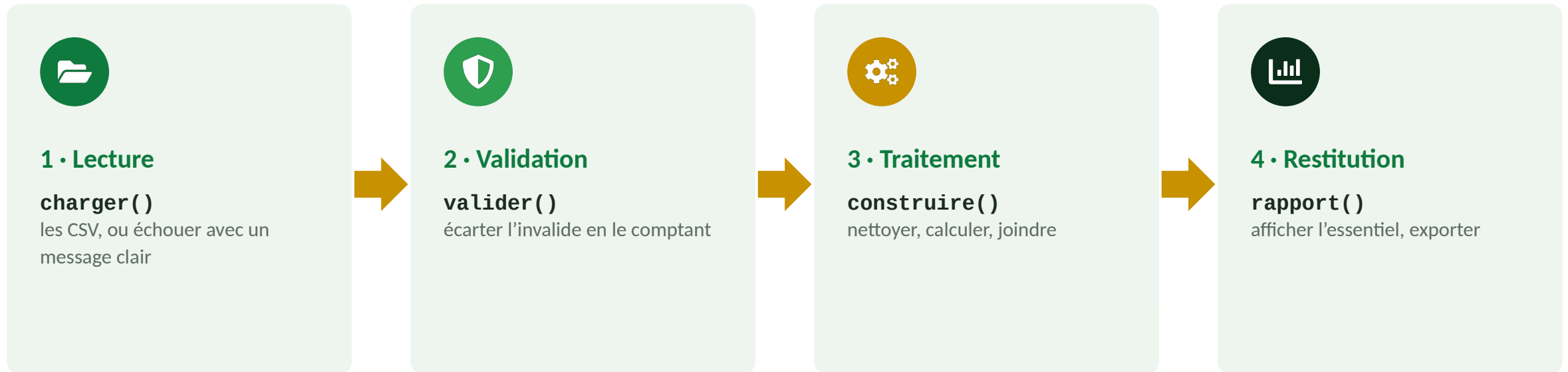
Objets & pandas

J5

Intégration & projets

Cet après-midi : les soutenances.

Quatre étapes, toujours les mêmes



10-integration/pipeline.py est entièrement rédigé et commenté : c'est le MODÈLE du projet. Chaque fonction fait UNE étape ; main() les enchaîne et raconte l'exécution.



```
uv run python pipeline.py --region BIS
```

```
|
```

```
uv run python pipeline.py --toutes
```

argparse : le programme devient un outil



Options nommées et documentées

help="..." sur chaque option : --help devient le mode d'emploi, gratuit.



type= et default= valident

type=int convertit ET refuse "abc" ; default documente le comportement standard.



Groupe mutuellement exclusif

--region OU --toutes, jamais les deux : le module refuse à votre place.



nargs et choices

choices=["BIS", "GAB", ...] borne les valeurs ; nargs="+" accepte plusieurs régions.



pipeline.py

```
parser = argparse.ArgumentParser(
    description="Rapport TVA.")
g = parser.add_mutually_exclusive_group(
    required=True)
g.add_argument("--region",
    choices=["BIS", "GAB", "BAF", ...])
g.add_argument("--toutes",
    action="store_true")
parser.add_argument("--seuil",
    type=int, default=0,
    help="TVA minimale affichée")
```

Échouer proprement, tracer ce qu'on écarte



Jamais de traceback brut

`sys.exit("message + remède")` : l'utilisateur sait quoi faire, code retour $\neq 0$ pour les scripts.



Chaque filtre annonce son bilan

« CA invalides écartés : 26 » — la donnée perdue est visible, donc contestable.



Le remède dans le message

« `→ python generate_data.py` » : l'erreur contient sa solution.



C'est un critère de la grille

Robustesse : 3 points sur 20 — vérifié en direct pendant la soutenance.



échec explicite

```
try:
    contr = pd.read_csv(...)
except FileNotFoundError as exc:
    sys.exit(
        f"Manquant : {exc.filename}\n"
        f"→ python generate_data.py")
```

```
$ python pipeline.py --toutes
```

```
1. Chargement...
```

```
CA invalides écartés      : 26
```

```
Dates inexploitable      : 0
```

```
$ echo $?      # 0 = succès
```

logging : print qui a grandi



Des niveaux, pas du bruit

DEBUG en développement, INFO en exploitation, WARNING pour l'anormal toléré.



On règle le volume SANS toucher au code

level=logging.INFO au lancement — les debug se taisent, le code reste.



Horodaté et attribuable

Le format ajoute l'heure et le module : indispensable dès qu'un script tourne la nuit.



print reste pour la restitution

Le RÉSULTAT s'affiche avec print ; le DÉROULEMENT se journalise avec logging.



logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s "
           "%(message)s")
log = logging.getLogger(__name__)

log.info("%s lignes lues", n)
log.warning("%s CA écartés", rejets)

2026-08-13 09:12:04 INFO 60000 lignes lues
```

Vectoriser plutôt que boucler



La boucle ligne à ligne ne tient pas

iterrows() sur 60 000 lignes : des secondes ; en colonne : des millisecondes.



Penser « colonne entière »

df["a"] - df["b"] applique l'opération partout — c'est l'esprit pandas.



np.where : le if vectorisé

np.where(cond, si_vrai, si_faux) remplace le if par ligne.



map pour les barèmes

df["regime"] = df["ca"].map(regime) : votre fonction du J1, appliquée en colonne.



colonne entière

```
# x lent : ligne à ligne
for i, ligne in df.iterrows():
    df.loc[i, "tva"] = ligne["ca"] * 0.18
```

```
# ✓ vectorisé : toute la colonne
df["tva"] = df["ca"] * 0.18
```

```
# le if vectorisé
df["etat"] = np.where(
    df["ca"] > 0, "OK", "KO")
```

```
df["regime"] = df["ca"].map(regime)
```

L'anatomie d'un script qui dure



Constantes en tête, en MAJUSCULES

DATA, FORMATS_DATE, TAUX_TVA : tout ce qui se règle, visible en 10 lignes.



Des fonctions courtes et nommées

Une étape = une fonction ; chaque docstring dit le POURQUOI.



main() raconte l'histoire

Quatre appels, quatre print numérotés — on suit l'exécution sans lire le détail.



Le garde __main__

Le fichier s'importe (pour les tests) ET se lance — sans double exécution.



squelette type

```
DATA = Path(__file__).parent / ...  
TAUX_TVA = 0.18  
  
def charger() -> ...: ...  
def valider(d) -> ...: ...  
def construire(...) -> ...: ...  
def rapport(...) -> None: ...  
  
def main() -> None:  
    # 1. 2. 3. 4. — et rien d'autre  
  
if __name__ == "__main__": main()
```


Le socle : vous n'écrivez que votre sujet

```
le squelette

projet-final/socle/projet.py

def charger():      # FOURNI
def valider():      # À VOUS DE JOUER
def traiter():      # À VOUS DE JOUER
def restituer():    # À VOUS DE JOUER
def main():         # FOURNI
```

Exécuté tel quel, le socle s'arrête sur `NotImplementedError` : c'est votre point de départ, pas un bug.



La structure est imposée

Quatre fonctions, un main : exactement `pipeline.py` — votre modèle.



Remplir dans l'ordre

valider → traiter → restituer → options `argparse` propres au sujet.



Interdits

versionner les données produites ; le copier-coller non compris (questions en soutenance).



README de groupe

quoi, comment lancer, qui a fait quoi — la base des questions individuelles.

La grille d'évaluation — 20 points

Critère	Points
La chaîne s'exécute de bout en bout sans erreur	6
Justesse : le résultat répond à la question du sujet	5
Robustesse : données sales gérées, messages clairs	3
Lisibilité : fonctions courtes, noms parlants, docstrings	3
Soutenance : clarté, démonstration, réponses	3



BONUS +1

un graphique matplotlib
pertinent dans le rapport

6 points sur 20 tiennent en une phrase : ÇA TOURNE. Sécurisez l'exécution avant d'embellir.

La soutenance : 10 + 5 minutes



2 min

Le sujet — La question métier, avec vos mots — pas de jargon.



5 min

La démonstration — UNE commande, en direct. Le bilan des lignes écartées commenté.



3 min

Un choix technique — Une difficulté rencontrée et comment vous l'avez tranchée.



5 min

Questions — Chacun répond sur SA partie — d'où le « qui a fait quoi » du README.

Conseil : testez la démonstration sur un poste PROPRE (clone frais + uv sync + génération des données) — c'est là que ça casse.

La checklist qualité — 10 minutes bien placées



Un clone frais tourne-t-il ?

git clone + uv sync + génération + UNE commande : le test de vérité.



Les rejets sont-ils tous comptés ?

Chaque filtre affiche son bilan — sinon une donnée disparaît en silence.



Les noms disent-ils le métier ?

taux_penalite, declarations_orphelines — pas x, tmp, df2.



Les docstrings disent-elles le POURQUOI ?

Le « quoi » se lit dans le code ; le « pourquoi » doit être écrit.



Rien de généré n'est versionné ?

git status propre : ni CSV produits, ni rapports, ni .venv.



répétition générale

```
# le test de vérité, en entier
$ git clone <depot-du-groupe> test
$ cd test && uv sync
$ cd 00-data
$ python generate_data.py --auto
$ cd ../projet
$ uv run python projet.py --region BIS
```

```
1. Chargement...
2. Validation... (rejets : 26)
3. Traitement...
4. Restitution... rapport.csv écrit
```

Continuer à progresser



Le dépôt reste

Solutions complètes, aide-mémoire (docs/), pipeline modèle : tout reste accessible et clonable.



Automatiser un vrai besoin

Choisissez UNE tâche répétitive de votre service et scriptez-la : c'est ainsi qu'on ancre.



Approfondir

pytest pour remplacer les assert, SQL pour dépasser le CSV, docs.python.org en français.

Prochaine marche naturelle : tests avec pytest, bases de données (SQL), environnements de production — le paquet fiscalite est prêt à grandir.

À retenir

- ✓ **Un environnement reproductible** — uv sync : la compétence invisible qui évite 80 % des « ça marche pas chez moi ».
- ✓ **Le réflexe de la donnée sale** — valider, compter ce qu'on écarte, échouer proprement, journaliser.
- ✓ **Du script au paquet** — fiscalite/ : votre code est devenu réutilisable et testable.
- ✓ **pandas vectorisé** — charger, nettoyer, joindre, agréger — en colonnes, pas en boucles.
- ✓ **Un projet soutenu** — une chaîne complète, exécutée en une commande, défendue en public.



Félicitations — et merci !

Le dépôt reste ouvert : git pull pour les dernières solutions. Vos retours nourriront la prochaine session.